

引用，指针，形参

形参

C++

```
1  #include <iostream>
2  using namespace std;
3
4  // 函数声明
5  void add(int x, int y);
6
7  int main() {
8      int a = 5, b = 7;
9      // 函数调用，实际参数 5 和 7 传递给形参 x 和 y
10     add(a, b);
11     return 0;
12 }
13
14 // 函数定义，形参 x 和 y 接收实际参数的值
15 void add(int x, int y) {
16     int sum = x + y;
17     cout << "Sum is: " << sum << endl;
18 }
```

引用交换

C++

```
1  //StudybarCommentBegin
2  #include <iostream>
3  using namespace std;
4  void swap(int &,int &); //定义的时候是引用，离开定义的时候是地址
5  int main( )
6  {
7      int i,j;
8      cin >> i >>j ;
9      swap(i,j);
10     cout<<i<<" "<<j<<endl;
11     return 0;
12 }
13 //StudybarCommentEnd
14 void swap(int &a,int &b) //别名
15 { int temp;
16   temp=a;
17   a=b;
18   b=temp;
19 }
```

指针交换

C++

```
1  //StudybarCommentBegin
2  #include <iostream>
3  using namespace std;
4  void swap(int *,int *);
5  int main( )
6  {
7      int i,j;
8      cin >> i >>j ;
9      swap(&i,&j);
10     cout<<i<<" "<<j<<endl;
11     return 0;
12 }
```

```
13 //StudybarCommentEnd
14 void swap(int *x,int *y)
15 {
16     int temp=*x;
17     *x=*y;
18     *y=temp;
19 }
```

引用与地址

C++

```
1 //在 C++ 中，引用通常不直接代表地址，而是作为源变量的别名。当我们创建一个引用时，
2 //编译器会将它关联到某个已存在的变量上，而不是直接将引用视为一个地址。
3 //虽然引用在内部可能实现为指针，但它隐藏了指针的复杂性，并提供了更简洁、更安全的语法。
4 //在引用被创建后，它就会一直指向其关联的变量，而不会改变或指向其他变量。
5 //与指针不同，引用不能改变自己所指向的变量。因此，一旦引用初始化后，
6 //它就不能再引用其他变量，这与指针是有区别的。
7 //下面是一个示例，演示了引用与地址的关系：
8 #include <iostream>
9 using namespace std;
10
11 int main() {
12     int x = 10;
13     int &ref = x; // 创建一个引用，关联到变量 x
14
15     cout << "Address of x: " << &x << endl; // 输出变量 x 的地址
16     cout << "Address of ref: " << &ref << endl; // 输出引用 ref 的地址
17
18     return 0;
19 }
```

引用介绍

引用是一种在程序中用来表示某个变量或对象的别名的概念。通过引用，可以使用一个名称来访问另一个变量或对象，而无需拷贝数据本身。引用提供了对变量或对象的间接访问方式。

C++

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 10;
6     int &ref = x; // 创建整型变量 x 的引用 ref
7     ref = 20;      // 修改引用 ref，也会改变 x 的值
8     cout << "x: " << x << endl; // 输出 x 的值
9     return 0;
10 }
```

指针是一种变量，它存储了另一个变量的内存地址。通过指针，可以直接访问或操作内存中的数据。指针提供了对内存的直接访问，因此在编程中具有重要的作用，例如动态内存分配、数据结构的实现等。

以下是 C++ 中的一个简单示例，演示了指针的基本用法：

C++

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 10; // 定义一个整型变量 x，值为 10
6     int *ptr;   // 定义一个整型指针 ptr
7
8     ptr = &x;   // 将指针 ptr 指向变量 x 的内存地址
9
10    cout << "x 的值为: " << x << endl; // 输出 x 的值
11    cout << "x 的内存地址为: " << &x << endl; // 输出 x 的内存地址
12    cout << "ptr 指向的值为: " << *ptr << endl; // 输出指针 ptr 指向的值
13    cout << "ptr 的值为: " << ptr << endl; // 输出指针 ptr 的值，即 x 的内存地址
14    return 0;
15 }
```